

EN UNA FRASE Sabes sustituir HTTP Basic por JWT: el cliente hace login una vez en `POST /auth/login`, recibe un token firmado con expiración y lo envía en `Authorization: Bearer <token>`. El servidor lo valida comprobando la firma, sin consultar la base de datos.

LO ESENCIAL

JWT	Cadena firmada y autocontenida con tres partes separadas por puntos: header, payload y signature, en Base64URL
HS256	HMAC con SHA-256: firma simétrica, el mismo secreto firma y verifica
Claims	Declaraciones del payload: <code>sub</code> (username), <code>iat</code> (emisión), <code>exp</code> (expiración) y la personalizada <code>rol</code>
Payload sin cifrar	Solo va codificado en Base64: no metas contraseñas ni datos sensibles en el token
jjwt-api, jjwt-impl, jjwt-jackson (0.12.6)	La API se usa al compilar; la implementación y la serialización JSON van con scope <code>runtime</code>
jwt.secreto / jwt.expiracion-ms	Secreto de firma (mínimo 32 caracteres ASCII) y duración del token (<code>3600000 ms = 1 hora</code>), en <code>application.properties</code>
Keys.hmacShaKeyFor(...)	Convierte el secreto textual en una <code>SecretKey</code>
JwtServicio	<code>generarToken(username, rol)</code> con <code>Jwts.builder()</code> ; <code>esValido</code> , <code>extraerUsername</code> y <code>extraerRol</code> con <code>Jwts.parser().esValido</code> devuelve <code>false</code> ante cualquier fallo: firma inválida, token manipulado o expirado
POST /auth/login (AuthController)	Endpoint público: busca la <code>Credencial</code> , compara con <code>PasswordEncoder.matches()</code> y devuelve <code>RespuestaToken</code>
PeticionLogin / RespuestaToken	Records de entrada y salida; la respuesta es <code>{"token": "...", "tipo": "Bearer"}</code>
BadCredentialsException	La lanza el propio <code>AuthController</code> si el usuario no existe o la contraseña no coincide: el cliente recibe 401
JwtFiltro (OncePerRequestFilter)	Se ejecuta una vez por petición: lee la cabecera <code>Bearer</code> , valida el token y rellena <code>SecurityContextHolder</code> con <code>ROLE_</code> + rol
addFilterBefore(jwtFiltro, UsernamePasswordAuthenticationFilter.class)	Coloca <code>JwtFiltro</code> antes del filtro estándar; sustituye a <code>httpBasic</code> en <code>SeguridadConfig</code>
SessionCreationPolicy.STATELESS	Sin sesiones HTTP: cada petición se autentica por su token

ERRORES TÍPICOS

1 **WeakKeyException: The specified key byte array is 80 bits...**

✓ el secreto es demasiado corto para HS256. Pon en `jwt.secreto` al menos 32 caracteres; puedes generar uno con `openssl rand -base64 32`.

2 **Petición rechazada sin explicación con un token expirado o manipulado**

✓ `esValido()` atrapa cualquier excepción y devuelve `false`. Si necesitas saber el motivo, añade un `log.debug` en el `catch` (con uno específico para `ExpiredJwtException`) sin cambiar el valor devuelto.

3 **POST /auth/login es rechazado antes de llegar al controlador**

✓ falta `.requestMatchers("/auth/**").permitAll()` en `authorizeHttpRequests`.

4 **Todos los endpoints protegidos rechazan la petición aunque el token sea bueno, sin errores en el log**

✓ has usado `addFilterAfter`. El filtro va antes: `addFilterBefore(jwtFiltro, UsernamePasswordAuthenticationFilter.class)`.

5 **NullPointerException en JwtFiltro al llamar a /libros sin token**

✓ comprueba `cabecera != null` antes de `cabecera.startsWith("Bearer ")`.

EN EL PROYECTO BIBLIOTECA

El paquete `seguridad` recibe `JwtServicio`, `JwtFiltro`, `AuthController`, `PeticionLogin` y `RespuestaToken`. `SeguridadConfig` (que sigue en `configuracion`) pierde `httpBasic`, hace público `/auth/**` y recibe `JwtFiltro` como parámetro del método `filterChain` para registrarlo con `addFilterBefore`. El `pom.xml` suma los tres artefactos de `jjwt` y `application.properties` las propiedades `jwt.secreto` y `jwt.expiracion-ms`, comunes a `dev` y `prod`. Solo queda pendiente Actuator + Docker (Capítulo 15).

■ CÓDIGO MÍNIMO Y PETICIONES

El login en `AuthController`: sin `AuthenticationManager`, solo repositorio y `PasswordEncoder`.

```
@PostMapping("/login")
public RespuestaToken login(@RequestBody PeticionLogin peticion) {
    Credencial credencial = credenciales.findByUsername(peticion.username())
        .filter(c → passwordEncoder.matches(peticion.password(), c.getPassword()))
        .orElseThrow(() → new BadCredentialsException("Usuario o contraseña incorrectos"));

    String token = jwtServicio.generarToken(credencial.getUsername(), credencial.getRol());
    return new RespuestaToken(token);
}
```

JAVA

El núcleo de `JwtFiltro.doFilterInternal`. Si no hay cabecera o el token no vale, la petición sigue sin autenticar y decide `SeguridadConfig`.

```
String cabecera = request.getHeader("Authorization");

if (cabecera != null && cabecera.startsWith("Bearer ")) {
    String token = cabecera.substring(7);

    if (jwtServicio.esValido(token)) {
        String username = jwtServicio.extraerUsername(token);
        String rol = jwtServicio.extraerRol(token);

        UsernamePasswordAuthenticationToken autenticacion = new UsernamePasswordAuthenticationToken(
            username, null, List.of(new SimpleGrantedAuthority("ROLE_" + rol)));
        SecurityContextHolder.getContext().setAuthentication(autenticacion);
    }
}

chain.doFilter(request, response);
```

JAVA

```
### 1. Login: devuelve {"token": "...", "tipo": "Bearer"}
POST http://localhost:8080/auth/login
Content-Type: application/json

{"username": "admin", "password": "admin123"}

### 2. Usar el token en un endpoint protegido
POST http://localhost:8080/libros
Authorization: Bearer PEGA_AQUI_EL_TOKEN
Content-Type: application/json

{"titulo": "Fahrenheit 451", "autor": "Ray Bradbury", "anio": 1953}
```

HTTP